

Самостоятельный учебный проект

Распознавание именованных сущностей в научных текстах

Сравнительный анализ архитектур на датасете SciERC

Иван Четвергов

chitiverg@gmail.com

Май 2026

Код проекта: <https://github.com/ivanchetvergov/SciNER>

Содержание

Введение	3
1 Данные	4
1.1 Датасет SciERC	4
1.2 Типы сущностей	4
1.3 BIO-разметка	4
1.4 Статистика	4
2 Архитектуры моделей	5
2.1 Энкодеры	5
2.2 Классификационные головы	5
2.3 Взвешивание классов	5
2.4 Полный список экспериментов	5
3 Препроцессинг данных	7
3.1 Общий пайплайн (оба эксперимента)	7
3.1.1 Схема разметки BIO	7
3.1.2 Токенизация и выравнивание меток	7
3.1.3 Динамический паддинг	7
3.2 Эксперимент 1 — базовый препроцессинг	7
3.3 Эксперимент 2 — улучшенный препроцессинг	7
3.3.1 Аугментация заменой сущностей (Entity Swap)	7
3.3.2 Межпредложенческий контекст	8
3.4 Взвешивание классов	8
3.5 Метрика оценки	8
4 Условия обучения	9
4.1 Гиперпараметры	9
4.2 Инфраструктура	9
5 Результаты	10
5.1 Эксперимент 1 — базовый препроцессинг	10
5.1.1 Влияние предобучения: SciBERT vs BERT vs RoBERTa	10
5.1.2 Взвешивание классов: noweight стабильно лучше	10
5.1.3 Архитектура головы: простое работает лучше	11
5.1.4 Анализ по типам сущностей	11
5.1.5 Анализ матриц ошибок	12
5.1.6 Кривые обучения	12
5.1.7 Сравнение с опубликованными результатами	13
5.2 Эксперимент 2 — улучшенный препроцессинг	13
Заключение	14

Введение

Распознавание именованных сущностей (Named Entity Recognition, NER) — базовая задача обработки естественного языка, которая состоит в выделении в тексте упоминаний объектов заданных категорий и их классификации. В научных текстах набор категорий существенно отличается от новостных или общелингвистических корпусов: вместо персон и организаций исследователям важны методы, метрики, датасеты и научные концепции.

Целью данного проекта является сравнительное исследование нейросетевых архитектур на задаче NER научных текстов. В качестве датасета использован SciERC [1] — корпус из 500 аннотированных научных абстрактов по компьютерным наукам. Набор содержит 6 типов именованных сущностей, что делает его нетривиальным: классы семантически близки и имеют существенный дисбаланс по частоте.

В работе решаются следующие задачи:

1. Реализация и обучение 10 моделей на базе предобученных трансформеров (BERT, RoBERTa, SciBERT) с различными классификационными головами.
2. Исследование влияния взвешивания классов, архитектуры головы и качества предобучения энкодера на результат.
3. Оценка влияния улучшенного препроцессинга данных: аугментации через замену сущностей и расширения контекста через соседние предложения.

1. Данные

1.1. Датасет SciERC

SciERC [1] — корпус научных абстрактов из 12 конференций по ИИ (ACL, EMNLP, NeurIPS и др.). Датасет содержит 500 документов, размеченных на уровне именованных сущностей, отношений и кореференций. В данной работе используется только задача NER.

Стандартное разбиение: 350 документов в обучающей выборке, 50 в валидационной, 100 в тестовой.

1.2. Типы сущностей

Датасет определяет 6 типов именованных сущностей (таблица 1).

Таблица 1: Типы сущностей в SciERC

Тип	Описание	Пример
Task	Задача или проблема	<i>machine translation</i>
Method	Метод, модель, система	<i>BERT, attention mechanism</i>
Metric	Метрика оценки	<i>F1 score, BLEU</i>
Material	Датасет, ресурс, материал	<i>Penn Treebank, ImageNet</i>
OtherScientificTerm	Прочий научный термин	<i>feature, embedding</i>
Generic	Общий термин, сложно категоризировать	<i>approach, model</i>

1.3. BIO-разметка

Сущности кодируются в формате BIO (Beginning–Inside–Outside):

- B-X — первый токен сущности типа X,
- I-X — продолжение сущности типа X,
- O — токен вне именованной сущности.

Итого 13 меток: O плюс B-/I- для каждого из 6 типов. Классы существенно несбалансированы: метка O составляет большинство токенов, а такие типы, как Metric и Task, встречаются значительно реже остальных.

1.4. Статистика

Дисбаланс классов — ключевая особенность датасета, влияющая на выбор функции потерь. В разделе 4 описано, как это учитывается при обучении.

2. Архитектуры моделей

Все модели построены по одной схеме: предобученный трансформерный энкодер принимает последовательность токенов и возвращает контекстуализированные представления размерности H ; поверх него ставится классификационная голова, которая предсказывает BIO-метку для каждого токена. Различия между моделями касаются либо выбора энкодера, либо архитектуры головы, либо того и другого.

2.1. Энкодеры

Все энкодеры — трансформеры с 12 слоями и $H = 768$, обученные самостоятельно на разных данных. Это позволяет сравнивать влияние доменного предобучения при фиксированной архитектуре.

BERT [2] (bert-base-cased) предобучен на корпусе общей тематики (BooksCorpus + Wikipedia). Он задаёт базовую линию: насколько хорошо работает общедоменная модель на специализированной задаче.

RoBERTa [3] (roberta-base) отличается от BERT более интенсивным обучением: больший корпус, динамическое маскирование и отказ от задачи предсказания следующего предложения (NSP). Ожидается, что эти изменения улучшат общее качество представлений, хотя отказ от NSP потенциально снижает чувствительность к межпредложенческим зависимостям.

SciBERT [4] (allenai/scibert_scivocab_cased) обучен на 1.14 млн научных статей из Semantic Scholar и имеет собственный научный словарь. Научные тексты содержат специфическую терминологию (названия методов, датасетов, метрик), которой нет в общедоменных корпусах, поэтому SciBERT должен давать существенное преимущество на SciERC.

2.2. Классификационные головы

Выбор головы определяет, насколько сложную функцию можно построить поверх представлений энкодера и учитываются ли зависимости между соседними метками.

Linear. Дропаут с вероятностью p (берётся из конфига энкодера), затем линейный слой $\mathbb{R}^H \rightarrow \mathbb{R}^K$, где $K = 13$. Это минимальное решение: всю работу по различению классов делает энкодер, голова только проецирует. Обучается с токенов-уровневой кросс-энтропией.

MLP. Дропаут $\rightarrow \text{Linear}(H, 256) \rightarrow \text{GELU} \rightarrow \text{Linear}(256, K)$. По сравнению с Linear добавляет нелинейный слой, который может улучшить разделимость классов в пространстве признаков. Гипотеза: если представления энкодера не вполне линейно разделимы, MLP компенсирует это.

Linear + CRF. Линейный слой формирует эмиссионные потенциалы, поверх которых CRF-слой [5] моделирует зависимости между соседними метками. Главное преимущество CRF — принудительная BIO-согласованность: декодирование по алгоритму Витерби никогда не выдаст I-X без предшествующего B-X. Обучение ведётся по отрицательному логарифмическому правдоподобию последовательности, что сложнее токенов-уровневой кросс-энтропии.

Concat-4. Вместо финального скрытого состояния берётся конкатенация выходов последних 4 слоёв энкодера: $\mathbb{R}^{4H}$, затем дропаут и линейный слой. Мотивация: нижние слои BERT захватывают морфологию и синтаксис, верхние — семантику; их совместное использование может быть полезно для задачи, требующей и синтаксических (граница сущности), и семантических (тип сущности) сигналов.

2.3. Взвешивание классов

Метка 0 составляет около 70% всех токенов. При равновесной кросс-энтропии модель может получать низкий лосс, по умолчанию предсказывая 0, при этом плохо распознавая редкие типы. Чтобы компенсировать дисбаланс, применяются веса классов:

$$w_c = \sqrt{\frac{N}{K \cdot n_c}},$$

где N — общее число токенов в обучающей выборке, n_c — число токенов класса c . Квадратный корень смягчает взвешивание: без него соотношение весов 0 и Metric достигало бы $35\times$, что чрезмерно штрафует частые классы и снижает точность. Sqrt снижает это соотношение примерно до $6\times$.

Каждая модель обучается в двух вариантах — с весами и без, — чтобы эмпирически проверить, помогает ли взвешивание на данном датасете.

2.4. Полный список экспериментов

Таблица 2: Модели, участвующие в эксперименте

Название	Энкодер	Голова	Веса
bert_linear_noweight	BERT	Linear	—
bert_linear	BERT	Linear	✓
roberta_linear_noweight	RoBERTa	Linear	—
roberta_linear	RoBERTa	Linear	✓
scibert_linear_noweight	SciBERT	Linear	—
scibert_linear	SciBERT	Linear	✓
scibert_mlp	SciBERT	MLP	—
scibert_linear_crf_noweight	SciBERT	Linear+CRF	—
scibert_linear_crf	SciBERT	Linear+CRF	✓
scibert_concat4	SciBERT	Concat-4	—

3. Препроцессинг данных

Проводится два эксперимента с разными условиями подготовки данных. Все изменения касаются только стороны данных — архитектуры и гиперпараметры одинаковы в обоих.

3.1. Общий пайплайн (оба эксперимента)

3.1.1. Схема разметки BIO

Сущности кодируются в формате BIO: B-X — первый токен сущности типа X, I-X — продолжение, O — вне сущности. Итого 13 меток.

Альтернатива — схема BIOES (добавляет S-X для одиночных и E-X для последних токенов). BIOES точнее кодирует границы и теоретически упрощает задачу для CRF, однако увеличивает пространство меток с 13 до 21. На практике для коротких span'ов, типичных для SciERC, выигрыша не наблюдается, поэтому выбран стандартный BIO.

3.1.2. Токенизация и выравнивание меток

WordPiece (BERT, SciBERT) и BPE (RoBERTa) разбивают слова на субтокены. Возникает вопрос: какую метку присваивать продолжающим субтокенам?

Вариант «-100»: продолжающие субтокены маскируются — модель учится только на первом субтокене каждого слова. Это стандартный подход в ранних реализациях. Недостаток: большая часть обучающего сигнала теряется; для длинных слов, характерных для научных текстов (*convolutional*, *bidirectional*), модель видит только первый фрагмент.

Вариант «I-X» (используется в обоих экспериментах): продолжающие субтокены получают метку I-X, если слово начинается с B-X, или ту же I-X, если слово уже было I-X. Это увеличивает количество обучающих примеров и делает предсказания более согласованными на уровне субтокенов. Трейдоф: для однотоковых сущностей, разбитых на несколько субтокенов, модель вынуждена предсказывать B-X на первом и I-X на остальных — технически правильно, но создаёт слабо мотивированный I-X без предшествующего полного слова.

3.1.3. Динамический паддинг

Паддинг выполняется не при токенизации, а при формировании батча: последовательности дополняются до длины самой длинной в батче. Преимущество: при типичных длинах предложений SciERC (20–60 токенов) паддинг до 512 означал бы, что 85–90% вычислений аттеншена тратится на паддинговые токены. Динамический паддинг устраняет эту неэффективность без потери качества.

3.2. Эксперимент 1 — базовый препроцессинг

Каждое предложение обрабатывается независимо, аугментация и межпредложенческий контекст не применяются.

3.3. Эксперимент 2 — улучшенный препроцессинг

3.3.1. Аугментация заменой сущностей (Entity Swap)

Мотивация. BERT-based NER-модели склонны запоминать поверхностные формы сущностей: если *LSTM* встречается в обучающей выборке как *Method*, модель может учиться на имени, а не на контексте. На маленьком датасете (350 документов) это особенно опасно. Entity swap нарушает эту корреляцию: заменяя сущность на другую того же типа, мы сохраняем синтаксический контекст, но меняем поверхностную форму, вынуждая модель опираться на окружение.

Реализация. По обучающей выборке строится словарь: для каждого типа сущностей собираются все встречающиеся span'ы. При аугментации каждый span в предложении заменяется с вероятностью $p = 0.5$ случайным span'ом того же типа. Замена только внутри типа принципиальна: межтиповая замена порождала бы противоречивые примеры (*Method* на месте *Metric* в том же синтаксическом контексте).

Трейдофы.

- Аугментированные предложения семантически могут быть неестественными (редкая метрика в контексте, типичном для метода). Это шум, но регуляризирующий.
- Датасет увеличивается до $\sim 2\times$ — обучение занимает вдвое больше времени при том же числе эпох.

- Для очень редких типов (Metric, Task) словарь мал, и разнообразие замен ограничено.

3.3.2. Межпредложенческий контекст

Мотивация. BERT кодирует каждое предложение независимо, не видя соседних. Однако в научных текстах именованные сущности часто вводятся и используются повторно в нескольких предложениях подряд. Граничные токены первого и последнего слова предложения лишены левого и правого контекста соответственно.

Реализация. Для каждого предложения s_i в документе к нему приписываются соседи с окном $w = 1$:

$$\underbrace{s_{i-1}}_{\text{контекст, метка}=-100} \quad \underbrace{s_i}_{\text{цель}} \quad \underbrace{s_{i+1}}_{\text{контекст, метка}=-100}$$

Токены контекстных предложений получают маску -100 : функция потерь не вычисляется по ним, но энкодер обрабатывает их наравне с целевыми. Контекст применяется к train, dev и test: это корректно, поскольку на инференсе соседние предложения также доступны.

Трейдофы.

- Длина входной последовательности увеличивается примерно втрое. При лимите 512 токенов предложения длиннее ~ 170 токенов с контекстом будут усечены. В SciERC такие случаи единичны.
- Окно $w > 1$ даёт больше контекста, но при $w = 2$ риск усечения центрального предложения существенно возрастает, а выигрыш от дальних предложений снижается.
- Вычислительные затраты растут квадратично по длине последовательности из-за механизма внимания.

3.4. Взвешивание классов

Мотивация. В SciERC метка 0 составляет около 70% всех токенов, тогда как Metric — около 2%. При стандартной кросс-энтропии модель может игнорировать редкие классы и получать низкий лосс, просто предсказывая 0 везде.

Обратная частота $w_c = N / (K \cdot n_c)$ исправляет это, но создаёт соотношение весов до $35\times$ между 0 и Metric — модель начинает чрезмерно «видеть» сущности, снижая точность.

Sqrt-сглаживание:

$$w_c = \sqrt{\frac{N}{K \cdot n_c}}$$

уменьшает соотношение примерно до $6\times$, балансируя между полным игнорированием дисбаланса и его гиперкоррекцией. Квадратный корень — эвристика без строгого обоснования; альтернатива — настройка коэффициентов вручную, что нецелесообразно при 13 классах.

Каждая модель обучается в вариантах с весами и без, чтобы эмпирически проверить, помогает ли взвешивание именно на этом датасете.

3.5. Метрика оценки

Качество оценивается по entity-level macro F1 (библиотека seqeval). Сущность считается правильно распознанной только при точном совпадении границ span'a и типа. Macro F1 усредняет F1 по всем 6 типам с равными весами, что делает метрику чувствительной к редким классам — именно здесь сосредоточена практическая сложность задачи.

Альтернатива — micro F1 — взвешивает по числу сущностей каждого типа, что даёт непропорционально большой вес частым типам (Generic, Method) и скрывает плохое качество на Metric и Task.

4. Условия обучения

4.1. Гиперпараметры

Все модели обучаются с едиными гиперпараметрами (таблица 3), чтобы различия в результатах можно было однозначно отнести к архитектуре, а не к условиям обучения.

Таблица 3: Гиперпараметры обучения

Параметр	Значение
Оптимизатор	AdamW, weight decay = 0.01
Learning rate	2×10^{-5} (энкодер + голова)
Learning rate (CRF)	10^{-3} (только CRF-слой)
Планировщик	Linear warmup (1 эпоха) + линейный decay
Batch size	16 (8 для scibert_concat4)
Максимум эпох	10
Early stopping patience	3 (по dev macro F1)
Случайное зерно	42
Смешанная точность	AMP fp16 на GPU

Выбор числа эпох. Лимит в 10 эпох — верхняя граница, реальным критерием остановки служит early stopping: обучение прекращается, если dev macro F1 не улучшается 3 эпохи подряд, и сохраняется лучший чек-поинт. На датасете SciERC (1500 обучающих предложений) BERT-scale модели эмпирически выходят на плато к 6–8 эпохе, а дальнейшее снижение train loss сопровождается ростом dev loss — признак переобучения. Увеличение лимита до 15–20 эпох не изменило бы результатов: early stopping остановил бы большинство прогонов раньше. В экспериментах большинство моделей остановились на 7–10 эпохе, что подтверждает достаточность выбранного лимита (рис. 4).

Выбор learning rate. Значение 2×10^{-5} — стандартное для fine-tuning BERT-scale энкодеров [2]. Для CRF-слоя используется отдельный, более высокий lr 10^{-3} : CRF не имеет предобученных весов и требует более быстрой начальной сходимости.

4.2. Инфраструктура

Обучение проводилось на платформе Kaggle на двух GPU NVIDIA T4 (по 15.6 GB памяти). Для параллелизма применялся `torch.nn.DataParallel`, за исключением `scibert_concat4`: эта модель захватывает скрытые состояния промежуточных слоёв через forward-хуки, что несовместимо с `DataParallel`. Тестовая оценка производится на чекпоинте лучшей эпохи по dev F1.

5. Результаты

5.1. Эксперимент 1 — базовый препроцессинг

Результаты всех моделей на тестовой выборке приведены в таблице 4.

Таблица 4: Macro F1 и per-entity F1 на тесте (эксперимент 1)

Модель	Macro	Gen.	Mat.	Met.	Metr.	OST	Task
bert_linear_noweight	0.604	0.714	0.557	0.681	0.573	0.531	0.565
bert_linear	0.594	0.656	0.568	0.676	0.593	0.523	0.546
roberta_linear_noweight	0.591	0.692	0.561	0.671	0.548	0.519	0.556
roberta_linear	0.585	0.673	0.557	0.651	0.593	0.531	0.505
scibert_linear_noweight	0.646	0.712	0.659	0.749	0.618	0.566	0.571
scibert_linear	0.603	0.688	0.615	0.725	0.533	0.545	0.515
scibert_mlp	0.644	0.708	0.665	0.746	0.615	0.577	0.553
scibert_linear_crf_noweight	0.628	0.721	0.628	0.723	0.543	0.570	0.581
scibert_linear_crf	0.628	0.721	0.628	0.723	0.543	0.570	0.581
scibert_concat4	0.644	0.737	0.661	0.751	0.573	0.579	0.561

Gen. = Generic, Mat. = Material, Met. = Method, Metr. = Metric, OST = OtherScientificTerm.

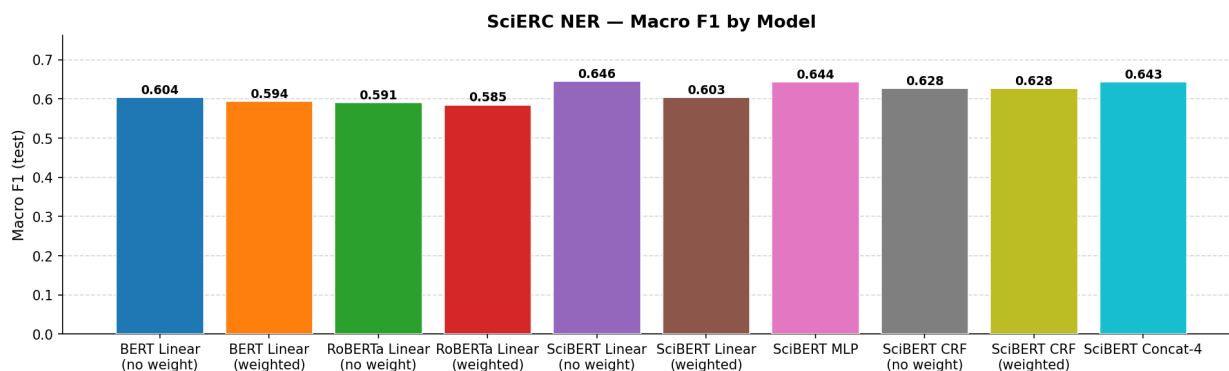


Рис. 1: Macro F1 по моделям (эксперимент 1)

5.1.1. Влияние предобучения: SciBERT vs BERT vs RoBERTa

Наиболее значимый фактор — качество предобучения энкодера. `scibert_linear_noweight` (0.646) превосходит лучший BERT (`bert_linear_noweight`, 0.604) на 4.2 п.п. и лучший RoBERTa (`roberta_linear_noweight`, 0.591) на 5.5 п.п.

Разрыв максимален на типах Material и Method — именно там, где доменная специфика проявляется сильнее всего. Material включает названия датасетов и ресурсов (*Penn Treebank*, *ImageNet*): SciBERT, обученный на научных статьях, видел эти упоминания многократно, тогда как BERT знает их лишь из Википедии. По Material SciBERT даёт 0.659 против 0.557 у BERT — разрыв 10 п.п.

RoBERTa несмотря на более агрессивное предобучение уступает BERT на большинстве типов. Возможная причина: RoBERTa обучена без NSP (Next Sentence Prediction), что снижает её чувствительность к межпредложенческим зависимостям. Кроме того, BPE-токенизация RoBERTa хуже справляется с редкими научными терминами по сравнению с WordPiece BERT.

5.1.2. Взвешивание классов: *noweight* стабильно лучше

Во всех парах вариант без взвешивания превосходит взвешенный: разрыв составляет от 0.6 п.п. (RoBERTa) до 4.3 п.п. (SciBERT). Это противоречит интуиции, но объясняется механизмом ошибок.

Взвешивание усиливает штраф за пропуск редких классов (Metric, Task), но одновременно снижает штраф за пропуск частых (Generic, Method, 0). На уровне отдельных типов видно: `scibert_linear` (взвешенный) имеет Generic 0.688 против 0.712 у `scibert_linear_noweight` и Method 0.725 против 0.749. Модель начинает «видеть» сущности там, где их нет, — растёт число ложных срабатываний на частых типах, что снижает их precision.

Выигрыш взвешивания на редких типах (Metric: +0.086 для BERT, +0.061 для RoBERTa) не компенсирует потерю на частых в рамках macro F1. Вывод: на данном датасете класс O настолько доминирует, что даже sqrt-сглаженное взвешивание сдвигает модель в сторону гиперчувствительности.

5.1.3. Архитектура головы: простое работает лучше

Среди SciBERT-моделей без взвешивания результаты следующие:

Голова	Macro F1
Linear	0.646
MLP	0.644
Concat-4	0.644
Linear+CRF	0.628

MLP и Concat-4 практически равны Linear ($\Delta < 0.002$). SciBERT-энкодер настолько мощный, что добавление нелинейности или многоуровневых признаков не даёт значимого прироста. При этом MLP и Concat-4 сложнее в обучении: у MLP больше параметров, у Concat-4 — нестандартный forward-pass, что потенциально увеличивает дисперсию.

CRF показывает худший результат среди SciBERT-моделей: 0.628, на 1.8 п.п. ниже линейной головы. Причин несколько. Во-первых, BERT-энкодер уже неявно моделирует зависимости между метками: контекстуализированные представления содержат информацию о соседних токенах, и BIO-согласованность возникает как следствие, а не должна задаваться явно. Во-вторых, CRF-loss оптимизирует правдоподобие всей последовательности целиком, что делает обучение менее стабильным на малом датасете. В-третьих, применение весов классов через умножение эмиссий (вместо корректного включения в маргинальное распределение CRF) является приближением.

5.1.4. Анализ по типам сущностей

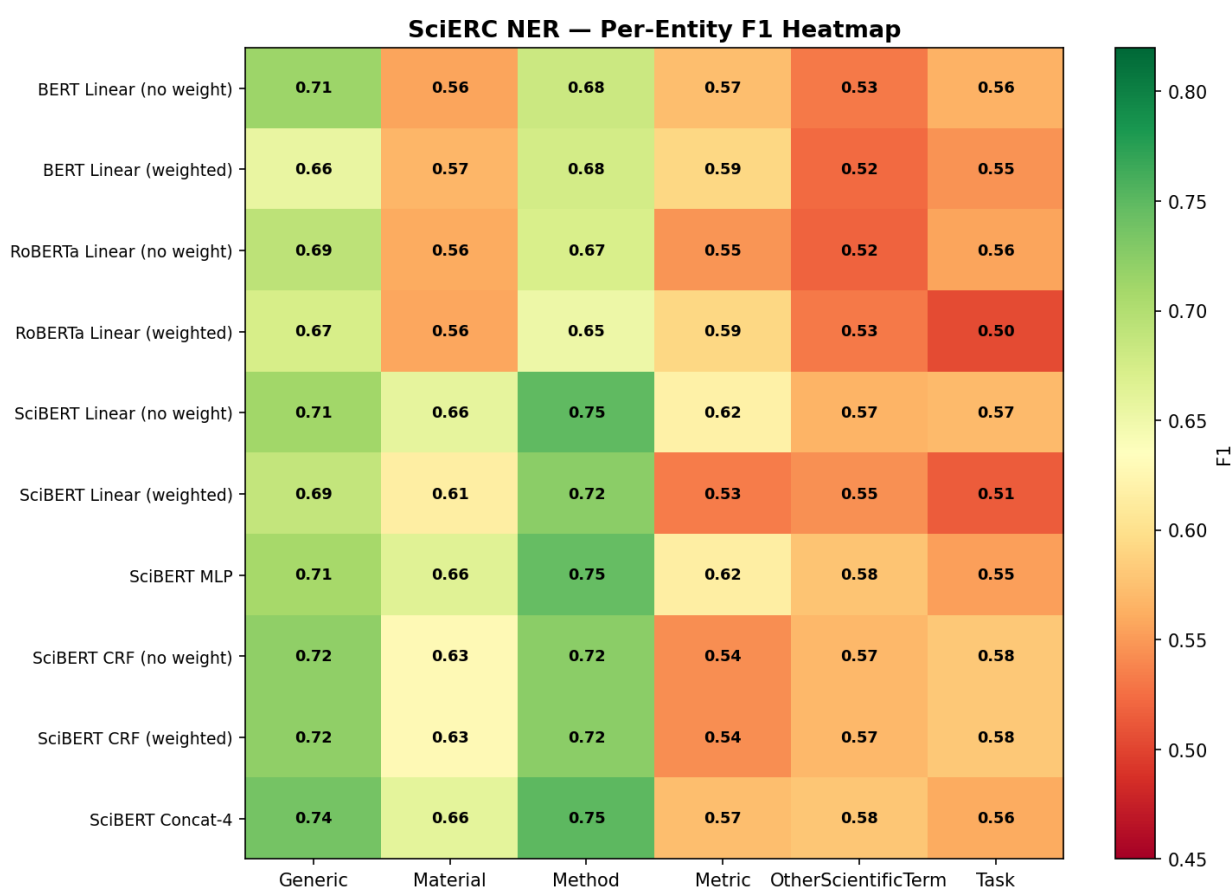


Рис. 2: Тепловая карта per-entity F1 (эксперимент 1)

Рисунок 2 позволяет выявить следующие закономерности.

Method — лучше всего распознаётся всеми моделями (0.65–0.75). Названия методов имеют отчётливые контекстные сигналы: «*we propose*», «*based on*», «*model*».

Generic — высокий F1 (0.66–0.74), хотя это «мусорная корзина» для трудно категоризируемых терминов. Объяснение: Generic-сущности частотны, модель видит достаточно примеров.

Material — высокая дисперсия (0.56–0.66). Наиболее чувствителен к доменному предобучению: именно здесь SciBERT отрывается от BERT/RoBERTa.

Metric — стабильно трудный тип (0.53–0.62). Многие метрики — короткие аббревиатуры (*F1*, *BLEU*, *ROUGE*), которые встречаются в разных контекстах и легко путаются с Generic или Method.

OtherScientificTerm — самый низкий F1 (0.52–0.58) по всем моделям. Это ожидаемо: OST — остаточная категория без чёткого семантического ядра, куда попадают все термины, не вошедшие в другие типы.

Task — 0.50–0.58, близко к OST. Описания задач разнообразны и часто многословны (*machine translation*, *named entity recognition*), что усложняет определение точных границ span’a.

5.1.5. Анализ матриц ошибок

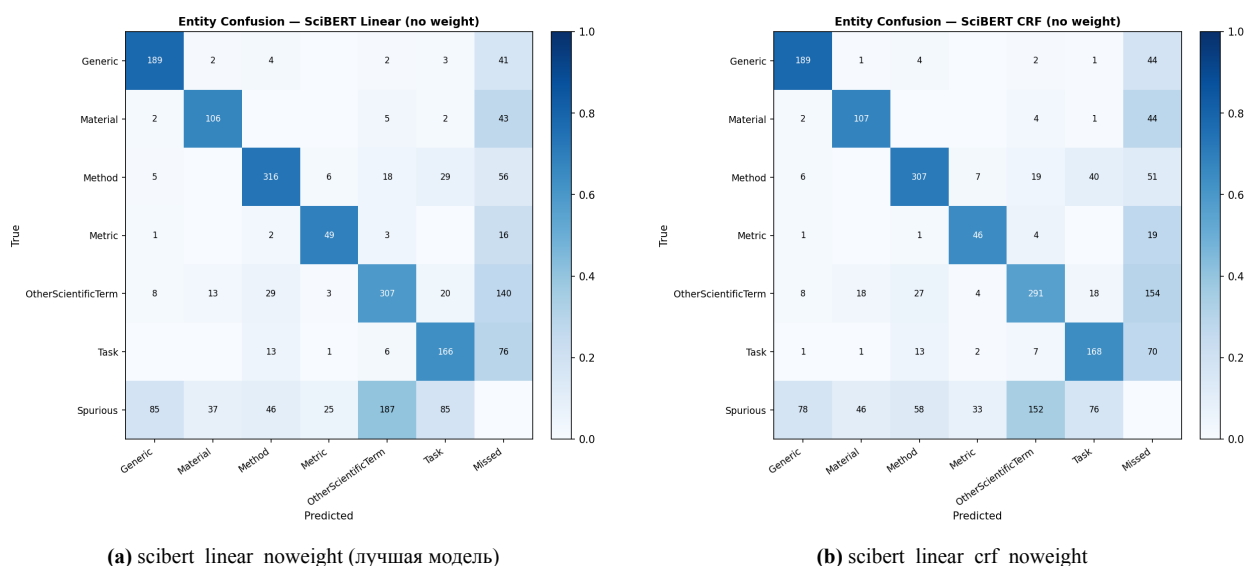


Рис. 3: Матрицы ошибок (нормировано по строкам; «Missed» = FN, «Spurious» = FP)

Матрицы строятся на уровне сущностей: строка соответствует истинному типу, столбец — предсказанному. Ячейка «Missed» — пропущенные сущности (FN), строка «Spurious» — ложные срабатывания (FP).

Ключевые наблюдения:

Пропуски доминируют над путаницей типов. Для всех моделей основная ошибка — не «Method принят за Generic», а «Method пропущен». Это означает, что модели скорее осторожны в предсказаниях, чем агрессивны. Такое поведение типично для BERT-based моделей без взвешивания.

Редкие типы пропускаются чаще. Metric и Task имеют наибольшую долю в столбце «Missed» — что согласуется с их низким F1 и редкостью в обучающей выборке.

Путаница Generic ↔ OtherScientificTerm. Это наиболее частая межтиповая ошибка, что семантически объяснимо: обе категории охватывают расплывчатые научные понятия без чётких лексических маркеров.

CRF не снижает число пропусков. Несмотря на то что CRF моделирует зависимости между метками, его матрица ошибок похожа на линейную голову. CRF улучшает BIO-согласованность (нет I-X без B-X), но не помогает с распознаванием трудных типов.

5.1.6. Кривые обучения

SciBERT-модели выходят на плато к эпохе 7–8, BERT — к 8–10. Это соответствует более высокому starting point SciBERT: он ближе к целевому распределению уже после первой эпохи флайн-тюнинга. RoBERTa демонстрирует заметную нестабильность на первых 3–4 эпохах: большой learning rate (тот же 2×10^{-5} , что и для остальных) может быть завышен для её архитектуры при таком малом датасете.

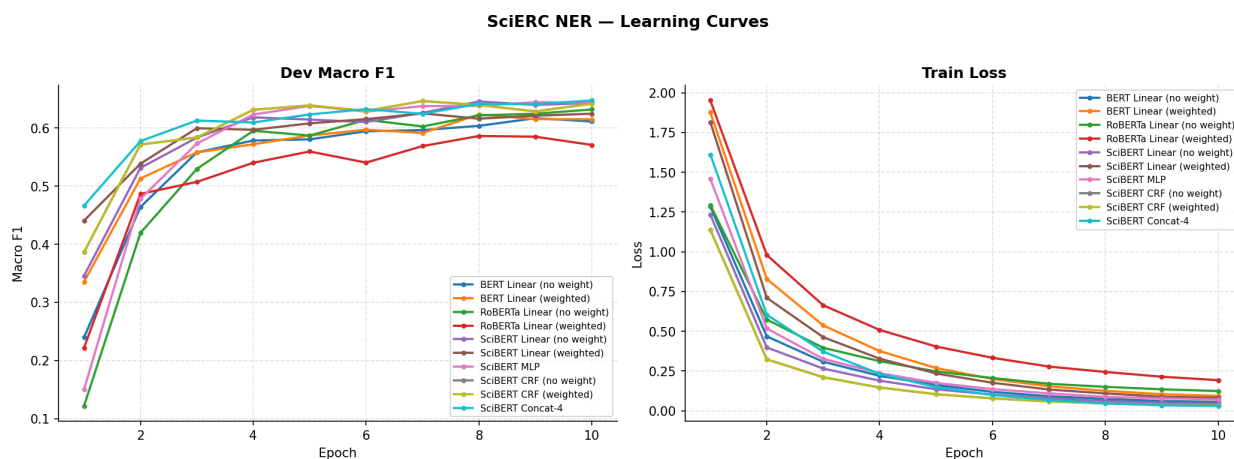


Рис. 4: Dev macro F1 и train loss по эпохам (эксперимент 1)

CRF-модели имеют более высокий начальный train loss и медленнее убывают — CRF оптимизирует правдоподобие последовательности, что труднее, чем токен-уровневая кросс-энтропия.

5.1.7. Сравнение с опубликованными результатами

На SciERC лучшие опубликованные результаты для задачи только NER достигаются span-based моделями: DyGIE++ [6] сообщает $F1 \approx 0.67\text{--}0.68$. Наш лучший результат (0.646) уступает примерно на 2–3 п.п.

Разрыв объясним методологически: span-based подходы напрямую классифицируют кандидатные span'ы, избегая ошибок выравнивания субтокенов, и часто обучаются совместно с задачей извлечения отношений (multi-task learning), что даёт дополнительный обучающий сигнал. Token-level BIO является упрощением, при котором ошибка в одном токене разрушает весь span.

5.2. Эксперимент 2 — улучшенный препроцессинг

Результаты будут добавлены после завершения обучения.

Заключение

В работе проведено сравнительное исследование 10 нейросетевых архитектур на задаче NER в научных текстах (датасет SciERC).

Основные результаты. Лучший результат в базовом эксперименте показала модель `scibert_linear_noweight` — макро F1 = 0.646. SciBERT стабильно превосходит BERT и RoBERTa за счёт научного предобучения. Простая линейная голова без взвешивания классов оказалась конкурентоспособнее сложных вариантов (MLP, CRF, Concat-4).

Сильные стороны.

- Контролируемое сравнение: все модели обучены с одинаковыми гиперпараметрами, единственная переменная — архитектура.
- Полный пайплайн: от данных до confusion matrix и кривых обучения.
- Воспроизводимость: фиксированный seed, реестр прогонов.

Слабые стороны.

- Одна случайная инициализация на модель — нет доверительных интервалов.
- CRF-модели дали идентичные результаты в двух вариантах (вероятно, технический артефакт Kaggle-сессии).
- Редкие типы (Metric, Task) остаются сложными для всех моделей.

Направления развития.

- Span-based модели (DyGIE++, SpanBERT) вместо token-level BIO — качественно другой подход к границам сущностей.
- Joint NER+RE обучение, поскольку SciERC содержит разметку отношений.
- Более крупные энкодеры (SciBERT-large, SPECTER).

Список литературы

- [1] Luan, Y., He, L., Ostendorf, M., & Hajishirzi, H. (2018). *Multi-Task Identification of Entities, Relations, and Coreference for Scientific Knowledge Graph Construction*. EMNLP 2018.
- [2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL 2019.
- [3] Liu, Y., Ott, M., Goyal, N., et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692.
- [4] Beltagy, I., Lo, K., & Cohan, A. (2019). *SciBERT: A Pretrained Language Model for Scientific Text*. EMNLP 2019.
- [5] Lafferty, J., McCallum, A., & Pereira, F. (2001). *Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data*. ICML 2001.
- [6] Wadden, D., Wennberg, U., Luan, Y., & Hajishirzi, H. (2019). *Entity, Relation, and Event Extraction with Contextualized Span Representations*. EMNLP 2019.